

Prueba Técnica – Desarrollador .NET

Contexto del reto

Outlet Rental Cars (<https://www.outletrentalcars.com>) busca implementar un **Sistema de Búsqueda de Vehículos** que permita a sus clientes encontrar opciones disponibles de forma eficiente, escalable y alineada con buenas prácticas de arquitectura.

El objetivo es construir una **Web API** que exponga un endpoint de búsqueda de vehículos, considerando reglas de negocio reales del dominio

Requerimiento Funcional

El sistema debe permitir buscar vehículos disponibles considerando los siguientes criterios:

1. Criterios de Búsqueda

El endpoint de búsqueda debe recibir como mínimo:

- **Localidad de recogida**
 - **Localidad de devolución**
 - **Fecha y hora de recogida**
 - **Fecha y hora de devolución**
- #### 2. Reglas de Negocio

La API debe aplicar las siguientes reglas:

1. **Disponibilidad por Localidad**
 - Un vehículo debe estar disponible en la localidad de recogida.
 - La localidad de devolución puede ser distinta.
2. **Disponibilidad por Mercado**
 - Solo se deben retornar vehículos habilitados para el país al que pertenece una localidad.
3. **Rango de Fechas**
 - Un vehículo no puede estar disponible si existe una reserva activa que se cruce con el rango de fechas solicitado.
4. **Estado del Vehículo**
 - Solo se deben retornar vehículos con estado Disponible.

Requerimiento Técnico

- Implementar una **Web API en .NET 7 o superior**.
- Aplicar **Clean Architecture**
- Uso correcto de **inyección de dependencias**.
- Aplicar principios **SOLID, DRY y KISS**.
- Utilizar:
 - **MySQL** para información transaccional (vehículos, reservas).
- **MongoDB** para información de configuración o catálogo (ej: mercados, tipos de vehículo).

No es obligatorio usar datos reales, se permite **data seed**.

- **Query** para la búsqueda de vehículos.
- **Command** para crear una reserva (mock o funcional).
- No es obligatorio usar buses de mensajería, pero sí separar responsabilidades.
- Al crear una reserva, generar un **evento de dominio** (ej: VehicleReservedEvent).
- El evento puede ser manejado de forma interna (in-memory).

Se deben incluir:

- **Pruebas unitarias**
 - Para reglas de negocio clave.
- **Pruebas de integración**
 - Para el endpoint de búsqueda.
- Uso de frameworks de testing comunes en .NET.

Entregables

1. Repositorio público en **GitHub** que contenga:
 - Código fuente.
 - README con:
 - Descripción de la solución.
 - Decisiones técnicas tomadas.
 - Cómo ejecutar el proyecto.
2. La aplicación debe poder ejecutarse localmente.